

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_0f586cf2-9eb\agent-aefdbd1110e5ca659.jsonl`  
- SHA-256: `944b90a30b886abb15d5fc9a9c9d762117e32da5fd97efeccc62f333238a045e`  
- Source modified: `2026-06-21T14:45:47+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_0f586cf2-9eb`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:42:18.599Z)

This is PR P0a from docs/PACKAGING_PLAN.md: introduce a resolved app-data HOME so a packaged app launched from an arbitrary CWD finds the same config/.env/DB/output, WITHOUT changing behaviour when RALLY_APP_HOME is unset. HARD CONTRACT: env unset => byte-identical to pre-P0a (CWD-relative). The repo: FastAPI engine, namespace package (no backend/__init__.py), hatchling editable install, windows-first users, requires-python>=3.10. Verified facts: full suite 1221 passed/7 skipped, ruff+mypy clean, coverage 85.10%, and an e2e proof showed config+DB land under RALLY_APP_HOME (nothing leaks to CWD) and == CWD when unset.

Changed/new files in the worktree C:/Users/avidu/Projects/bhi-wt-p0a:

```
- NEW backend/utils/app_home.py (the resolver:  
app_home/app_path/default_config_path/default_env_path/resolve_output_dir/default_os_app_home)  
- backend/config/loader.py (DEFAULT_CONFIG_PATH alias + use default_config_path())  
- backend/main.py (load_dotenv prefers app-home .env; static mount package-relative; output_dir  
via resolve_output_dir at server-start + trim)  
- scripts/doctor.py (writes config to default_config_path() not REPO_ROOT)  
- NEW tests/test_app_home.py
```

LENS: behaviour-neutrality

Read the changed files. ADVERSARIALLY verify the HARD CONTRACT that with RALLY_APP_HOME unset the behaviour is byte-identical to before. Check: does app_home()==Path(".") make default_config_path()==Path("config.json") and resolve_output_dir("output")=="output" on BOTH posix and Windows? Could the DEFAULT_CONFIG_PATH alias (evaluated at import) ever differ from the dynamic default_config_path() and confuse a reader? Does load_config still read the repo's config.json when run from repo root? Try to find ANY input where unset behaviour changed.

Read the actual files under C:/Users/avidu/Projects/bhi-wt-p0a. Return findings with severity blocker|major|minor|nit, each with file:line and a concrete fix. Be harsh but precise; do not invent issues.

2. assistant (2026-06-21T14:42:21.299Z)

I'll review this PR adversarially, focusing on the behaviour-neutrality contract. Let me start by reading the changed files.

3. user (2026-06-21T14:42:22.247Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.  
2  
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,  
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and  
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it  
6      from  
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user  
8      silently
```

```

7 "loses" their index. This module introduces ONE resolver every state path roots on.
8
9 Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10 * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11 * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13 The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/local/share``)
14 is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15 calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16 this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17 real user profile during a test run).
18
19 stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20 the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21 """
22
23 from __future__ import annotations
24
25 import os
26 import platform
27 from pathlib import Path
28
29 #: Environment variable the packaged launcher sets to pin the app-data home.
30 APP_HOME_ENV = "RALLY_APP_HOME"
31
32 #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33 APP_DIR_NAME = "Khelsutra"
34
35
36 def app_home() -> Path:
37     """The resolved application-data home.
38
39     ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40 to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41     """
42     env = os.environ.get(APP_HOME_ENV)
43     if env and env.strip():
44         return Path(env.strip()).expanduser()
45     return Path(".")
46
47
48 def app_path(*parts: str | os.PathLike[str]) -> Path:
49     """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50     return app_home().joinpath(*[str(p) for p in parts])
51
52
53 def default_config_path() -> Path:
54     """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56     With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57     """
58     return app_home() / "config.json"
59
60

```

```

61     def default_env_path() -> Path:
62         """`<app-home>/env` ? the dotenv file the server prefers (e.g.
63         ``GEMINI_API_KEY``)."""
64         return app_home() / ".env"
65
66     def resolve_output_dir(output_dir: str) -> str:
67         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
68         through.
69         With ``RALLY_APP_HOME`` unset, ``resolve_output_dir("output") == "output"`` (CWD-
70         relative),
71         byte-identical to today; with it set, ``"output"`` becomes ``<app-home>/output`` so
72         the SQLite
73         DB and produced reels live alongside the rest of the app's state.
74         """
75         if not output_dir:
76             return str(app_home() / "output")
77         if os.path.isabs(output_dir):
78             return output_dir
79         return str(app_home() / output_dir)
80
81     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
82         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
83         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
84
85         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
86         * macOS ? ``~/Library/Application Support/<app_name>``
87         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/local/share/<app_name>`` if
88         unset)
89         """
90         system = platform.system()
91         if system == "Windows":
92             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
93         elif system == "Darwin":
94             base = str(Path.home() / "Library" / "Application Support")
95         else:
96             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
97         return Path(base) / app_name

```

4. user (2026-06-21T14:42:22.788Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig

```

```

22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Back-compat alias (was a module constant; no external consumers). Prefer
33     # ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a).
With the
34     # env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
35     DEFAULT_CONFIG_PATH = default_config_path()
36
37
38     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
39         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
40         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
41         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
42         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
43         out = dict(base)
44         for key, val in override.items():
45             if isinstance(val, dict) and isinstance(out.get(key), dict):
46                 out[key] = _deep_merge(out[key], val)
47             else:
48                 out[key] = val
49         return out
50
51
52     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
53         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
54         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
55         environment never silently selects a profile (and never silently spends on cloud).
56
57         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
58         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
59         asserting a preset that was never applied. A bad value in config.json is left to
surface as
60         a hard, typed-Literal error downstream (config-file correctness)."""
61         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
62         if env_profile and env_profile.strip():
63             ep = env_profile.strip()
64             if is_known_profile(ep):
65                 return ep
66             logger.warning(
67                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
68                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
69             )
70             # fall through to the file's profile (env typo must not suppress a valid file
choice)
71         dep = file_data.get("deployment")
72         if isinstance(dep, dict):
73             prof = dep.get("profile")
74             if isinstance(prof, str) and prof.strip():
75                 return prof.strip()

```

```

76         return ""
77
78
79     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
80     "") -> list[str]:
81         """Dotted paths in `data` that the typed config will not honor.
82
83         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
84         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
85         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
86         user's intent is lost ? collect every such key so load_config can warn.
87         """
88         unknown: list[str] = []
89         fields = model_cls.model_fields
90         for key, val in data.items():
91             if key not in fields:
92                 unknown.append(prefix + key)
93                 continue
94             ann = fields[key].annotation
95             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
96                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
97         return unknown
98
99     def _get_builtin_defaults() -> dict[str, Any]:
100         """Return a plain dict of the current built-in defaults.
101
102         The AppConfig model is the single source of truth for defaults.
103         """
104         return AppConfig().model_dump(mode="json")
105
106
107     def load_config(path: str | Path | None = None) -> AppConfig:
108         """Load configuration with the following precedence:
109
110         1. Built-in defaults defined in the Pydantic models.
111         2. Values from the JSON file (if present and readable).
112         3. (Future) environment / CLI overrides.
113
114         Missing file or unreadable file ? returns a fully defaulted AppConfig.
115         Unknown keys in the file are tolerated (extra="allow" on the model).
116         """
117         cfg_path = Path(path) if path is not None else default_config_path()
118
119         file_data: dict[str, Any] = {}
120         if cfg_path.exists():
121             try:
122                 with open(cfg_path, "r", encoding="utf-8") as f:
123                     file_data = json.load(f)
124             except Exception as exc:
125                 # Non-fatal: continue with defaults + whatever we could parse.
126                 # In production we might log here.
127                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
128
129         # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
130         # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
131         # non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults +
warn, per the
132         # loader's "bad input is non-fatal" contract.
133         if not isinstance(file_data, dict):
134             logger.warning("%s is not a JSON object (got %s); ignoring it and using

```

```

defaults.",
135         cfg_path, type(file_data).__name__)
136     file_data = {}
137
138     if file_data:
139         unknown = _unknown_key_paths(file_data, AppConfig)
140         if unknown:
141             logger.warning(
142                 "[CONFIG WARNING] %s contains keys the typed config does not "
143                 "recognize (typo'd or removed knobs ? top-level extras are kept "
144                 "but unread; unknown section keys are silently dropped): %s",
145                 cfg_path, ", ".join(sorted(unknown)),
146             )
147
148     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
149     # at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
153     # unchanged.
154     defaults = _get_builtin_defaults()
155     profile = _resolve_explicit_profile(file_data)
156
157     if profile and not is_known_profile(profile):
158         # Only an unrecognised value from config.json reaches here (env typos already
159         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
160         # typed Literal at model_validate below, a hard clear error ? loud, never
161         # silent.
162         logger.warning(
163             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
164             preset.",
165             profile, ", ".join(PROFILE_PRESETS),
166         )
167         profile = ""
168
169     if profile:
170         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
171         # Record the profile actually applied ? the env may have chosen it over
172         # config.json,
173         # so re-stamp it onto the merged deployment section to keep the record honest.
174         merged.setdefault("deployment", {})
175         if isinstance(merged["deployment"], dict):
176             merged["deployment"]["profile"] = profile
177             ct = merged["deployment"].get("compute_target")
178             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
179             if not ct and canonical:
180                 # An active profile with no explicit compute_target (e.g. config.json
181                 # blanked it
182                 # to "") gets its canonical target back ? the record never silently lies
183                 # about an
184                 # active profile's compute_target (it stays meaningful for the M2+
185                 # ComputeBackend).
186                 merged["deployment"]["compute_target"] = canonical
187             elif ct and canonical and ct != canonical:
188                 logger.warning(
189                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
190                     "
191                     "config.json overrides it to %r ? honoring the explicit override.",
192                     profile, canonical, ct,
193                 )
194             logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",

```

```

186             profile, merged["deployment"].get("compute_target"))
187     else:
188         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
189         merged = {**defaults, **file_data}
190         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
191         if suggestion:
192             logger.debug(
193                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
194                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
195                 "auto-applies ? D15).", suggestion,
196             )
197
198     return AppConfig.model_validate(merged)
199
200
201 def save_default_config(path: str | Path | None = None) -> Path:
202     """Write a fresh default config.json.
203
204     Used by scripts/doctor.py diagnostics and the --setup CLI path.
205     Overwrites if the file exists.
206     """
207     cfg_path = Path(path) if path is not None else default_config_path()
208     cfg = AppConfig()
209     cfg_path.parent.mkdir(parents=True, exist_ok=True)
210     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
211     return cfg_path
212
213
214 # Backwards-compatible alias used by some older call sites during transition.
215 def get_default_config() -> dict[str, Any]:
216     """Return defaults as a plain dict (for transitional use only)."""
217     return _get_builtin_defaults()
218

```

5. user (2026-06-21T14:42:23.243Z)

```

1     """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3     The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
identical to the
4     pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
one home.
5     """
6
7     import os
8     from pathlib import Path
9
10    import pytest
11
12    from backend.utils import app_home as ah
13    from backend.config import loader
14
15
16    @pytest.fixture(autouse=True)
17    def _clear_app_home_env(monkeypatch):
18        """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
19        monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
20
21
22    # ---- default (env unset) is byte-identical to today ----
23

```

```

24     def test_app_home_defaults_to_cwd():
25         assert ah.app_home() == Path(".")
26
27
28     def test_default_config_path_unset_is_cwd_config_json():
29         # Pre-P0a this was the module constant Path("config.json").
30         assert ah.default_config_path() == Path("config.json")
31
32
33     def test_default_env_path_unset_is_cwd_dotenv():
34         assert ah.default_env_path() == Path(".env")
35
36
37     def test_resolve_output_dir_unset_is_identity_for_relative():
38         assert ah.resolve_output_dir("output") == "output"
39         assert ah.resolve_output_dir("custom") == "custom"
40
41
42     # ---- RALLY_APP_HOME set ? everything roots there ----
43
44     def test_app_home_honors_env(monkeypatch, tmp_path):
45         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
46         assert ah.app_home() == tmp_path
47
48
49     def test_app_home_expands_user(monkeypatch):
50         monkeypatch.setenv(ah.APP_HOME_ENV, "~/khealsutra-data")
51         assert ah.app_home() == Path.home() / "khealsutra-data"
52
53
54     def test_app_home_ignores_blank_env(monkeypatch):
55         monkeypatch.setenv(ah.APP_HOME_ENV, " ")
56         assert ah.app_home() == Path(".")
57
58
59     def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
60         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
61         assert ah.default_config_path() == tmp_path / "config.json"
62         assert ah.default_env_path() == tmp_path / ".env"
63         assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
64
65
66     def test_resolve_output_dir_set(monkeypatch, tmp_path):
67         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
68         assert ah.resolve_output_dir("output") == str(tmp_path / "output")
69
70
71     def test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
72         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
73         abs_dir = str(tmp_path / "elsewhere") if os.path.isabs(str(tmp_path)) else
os.path.abspath("elsewhere")
74         assert ah.resolve_output_dir(abs_dir) == abs_dir
75
76
77     def test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
78         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
79         assert ah.resolve_output_dir("") == str(tmp_path / "output")
80
81
82     # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----
83
84     def test_default_os_app_home_windows(monkeypatch, tmp_path):
85         monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
86         monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))

```



```

87         assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"
88
89
90     def test_default_os_app_home_macos(monkeypatch):
91         monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
92         assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"
93
94
95     def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
96         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
97         monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
98         assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"
99
100
101     def test_default_os_app_home_linux_default(monkeypatch):
102         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
103         monkeypatch.delenv("XDG_DATA_HOME", raising=False)
104         assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
105
106
107     # ---- loader integration: load_config() roots on the app-home ----
108
109     def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
110         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
111         (tmp_path / "config.json").write_text('{"sport": "tennis"}', encoding="utf-8")
112         cfg = loader.load_config() # no explicit path ? default_config_path() under the
home
113         assert cfg.sport == "tennis"
114
115
116     def test_loader_no_config_in_home_uses_defaults(monkeypatch, tmp_path):
117         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path)) # empty dir, no config.json
118         cfg = loader.load_config()
119         # Defaults load cleanly (no crash, no file) ? same contract as a missing CWD
config.json.
120         assert cfg is not None
121         assert hasattr(cfg, "indexing")
122
123
124     def test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
125         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
126         written = loader.save_default_config() # no explicit path
127         assert written == tmp_path / "config.json"
128         assert written.exists()
129
130
131     def test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):
132         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
133         explicit = tmp_path / "explicit.json"
134         explicit.write_text('{"sport": "pickleball"}', encoding="utf-8")
135         cfg = loader.load_config(str(explicit))
136         assert cfg.sport == "pickleball"
137

```

6. assistant (2026-06-21T14:42:26.218Z)

Now let me read the other changed files: `backend/main.py` and `scripts/doctor.py`.

7. user (2026-06-21T14:42:27.301Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil

```

```

5
6 # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7 # root (requirements.txt, config.json). Resolve the root from this file's
8 # location so it works regardless of the caller's cwd.
9 REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11 def check_command(cmd):
12     """Check if a system command is available."""
13     return shutil.which(cmd) is not None
14
15 def run_pip_install():
16     """Install required Python packages."""
17     print("Installing Python packages from requirements.txt...")
18
19     # Check if NVIDIA GPU is present
20     has_gpu = check_command("nvidia-smi")
21
22     requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23     pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25     import platform
26     if has_gpu:
27         print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
28         pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
29     elif platform.system() == "Darwin":
30         # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
31         print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
32     else:
33         print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
34         pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
35
36     try:
37         subprocess.check_call(pip_cmd)
38         print("[SUCCESS] Python packages installed.")
39         return True
40     except subprocess.CalledProcessError as e:
41         print(f"[ERROR] Failed to install Python packages: {e}")
42         return False
43
44 def init_default_config():
45     """Initialize a default config.json if it doesn't exist.
46
47     Now delegates to the canonical backend.config loader so that defaults
48     stay in sync with the Pydantic models (single source of truth).
49     """
50     try:
51         from backend.config import save_default_config
52         from backend.utils.app_home import default_config_path
53     except Exception as e:
54         print(f"[ERROR] Could not import backend.config loader: {e}")
55         print("[INFO] Falling back to no-op for config initialization.")
56         return
57
58     # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
59     # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root
while
60     # the server read from the CWD, so --setup and the server disagreed once CWD ? repo
root.
61     config_path = str(default_config_path())
62     if os.path.exists(config_path):
63         print(f"[INFO] {config_path} already exists. Skipping initialization.")
64         return
65

```

```

66         try:
67             saved_path = save_default_config(config_path)
68             print(f"[SUCCESS] config.json initialized with default settings at
{saved_path}.")
69         except Exception as e:
70             print(f"[ERROR] Failed to create config.json via loader: {e}")
71
72     def run_diagnostics():
73         """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
74         print("-" * 50)
75         print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
76         print("-" * 50)
77
78         # 1. Check Python Version
79         print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
80         if sys.version_info.major < 3 or (sys.version_info.major == 3 and
sys.version_info.minor < 8):
81             print("[WARNING] Python 3.8+ is highly recommended!")
82         else:
83             print("[PASS] Python version check passed.")
84
85         print("-" * 50)
86
87         # 2. Check FFmpeg / FFprobe
88         ffmpeg_ok = check_command("ffmpeg")
89         ffprobe_ok = check_command("ffprobe")
90
91         if ffmpeg_ok:
92             print("[PASS] FFmpeg is installed and available in PATH.")
93         else:
94             print("[FAIL] FFmpeg is NOT found in your PATH.")
95
96         if ffprobe_ok:
97             print("[PASS] FFprobe is installed and available in PATH.")
98         else:
99             print("[FAIL] FFprobe is NOT found in your PATH.")
100
101         if not (ffmpeg_ok and ffprobe_ok):
102             import platform
103             print("\n" + "!" * 50)
104             print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
105             _sys = platform.system()
106             if _sys == "Windows":
107                 print("Install on Windows: winget install Gyan.FFmpeg")
108             elif _sys == "Darwin":
109                 print("Install on macOS: brew install ffmpeg")
110             else:
111                 print("Install on Linux: sudo apt-get install -y ffmpeg (or your
distro's package manager)")
112             print("Or download a build from https://ffmpeg.org/download.html and put
ffmpeg/ffprobe on PATH.")
113             print("!" * 50 + "\n")
114         else:
115             print("[SUCCESS] All essential media command line tools are ready.")
116
117         print("-" * 50)
118
119         # 3. Check ML / GPU (Optional but good)
120         try:
121             import torch
122             cuda_ok = torch.cuda.is_available()
123             if cuda_ok:
124                 print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")

```

```

125         else:
126             print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
127         except ImportError:
128             print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
129
130         print("=" * 50)
131         return ffmpeg_ok and ffprobe_ok
132
133     if __name__ == "__main__":
134         init_default_config()
135         media_ok = run_diagnostics()
136         run_pip_install()
137         print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
138

```

8. user (2026-06-21T14:42:27.607Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10     # PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins
RALLY_APP_HOME),
11     # else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
12     # stdlib-only import ? keeps this pre-pydantic line light.
13     from backend.utils.app_home import default_env_path, resolve_output_dir
14
15     _env_path = default_env_path()
16     if _env_path.exists():
17         load_dotenv(_env_path)
18     else:
19         load_dotenv()
20
21     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
22     logging.basicConfig(
23         level=logging.INFO,
24         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
25         datefmt='%H:%M:%S'
26     )
27     logger = logging.getLogger(__name__)
28     from fastapi import FastAPI, HTTPException, Request
29     from fastapi.middleware.cors import CORSMiddleware
30     from pydantic import BaseModel
31     from typing import List, Dict, Any, Optional
32
33     from fastapi.staticfiles import StaticFiles
34     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
35     from backend.pipeline.validators.base import registry
36     from backend.infrastructure.database import Database
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.stitcher.compiler import VideoCompiler
39     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
40     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
41     from backend.config import ( # new typed config
42         load_config as load_app_config,
43         AppConfig,

```

```

44     StitchingConfig,
45     enforce_startup_checks,
46     StartupCheckError,
47 )
48 from backend.results import Failure # standardized error/result contract
49 from backend.reporting import emit_indexer_report
50 from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
51 from backend import billing # M8: per-job cost ledger (USD internal / INR display)
52 from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
53
54 app = FastAPI(title="Sports Highlight Indexer API")
55
56
57 def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
58     """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
59     default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
60     does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
61     src = os.environ if env is None else env
62     raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
63     origins = [o.strip() for o in raw.split(",") if o.strip()]
64     return origins or ["*"]
65
66
67 # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
68 # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
69 # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
70 # lock it to its origin; the middleware is fixed at startup.
71 app.add_middleware(
72     CORSMiddleware,
73     allow_origins=_cors_origins_from_env(),
74     allow_credentials=False,
75     allow_methods=["*"],
76     allow_headers=["*"],
77 )
78
79 # Serves static frontend files directly (now under api/ per approved structure).
80 # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
81 # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
82 # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
83 _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
84 os.makedirs(_STATIC_DIR, exist_ok=True)
85 app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
86
87 @app.get("/", response_class=HTMLResponse)
88 def serve_index():
89     index_path = os.path.join(_STATIC_DIR, "index.html")
90     if os.path.exists(index_path):
91         with open(index_path, "r", encoding="utf-8") as f:
92             return f.read()
93     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
94
95
96 # Global variables initialized at startup
97 db_instance: Optional[Database] = None
98 global_config: Optional[AppConfig] = None # now typed (Pydantic model)

```

```

98
99     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
100     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
101     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
102     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
103     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
104     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
105     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
106     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
107         JobWaiting,
108         _arm_wake_timer,
109         _drain_due_jobs,
110         _get_executor,
111         _job_to_request,
112         _MAX_DRAIN_WAKE_SEC,
113         _maybe_record_job_cost,
114         _run_claimed_job,
115         _schedule_next_drain_if_waiting,
116     )
117
118     # Legacy thin wrapper for any remaining direct calls during transition.
119     # New code should import load_app_config / AppConfig from backend.config directly.
120     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
121         cfg = load_app_config(config_path)
122         return cfg.model_dump(mode="json")
123
124     # --- API Models ---
125     class ProcessRequest(BaseModel):
126         video_path: str
127         player_pool: Optional[List[str]] = None
128         max_frames: Optional[int] = None
129         segmenter: Optional[str] = None
130
131     class QueryRequest(BaseModel):
132         video_id: str
133         server: Optional[str] = None
134         receiver: Optional[str] = None
135         duration_min: Optional[float] = None
136         event_type: Optional[str] = None
137
138     class CompileRequest(BaseModel):
139         video_id: str
140         segment_ids: List[int]
141         output_filename: str
142
143     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
144         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
145
146         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
147         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
148         and
149         keep the prior good results intact ? a failed/empty re-run never destroys prior
150         data.
151         """
152         if segments:
153             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
154         else:
155             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
156
157     # --- API Endpoints ---
158     @app.get("/api/config")

```

```

158     def get_config():
159         return global_config
160
161     @app.get("/api/health")
162     def health():
163         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
164         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
165         confirm the engine is reachable and which detector is wired. Never raises."""
166         sport = getattr(global_config, "sport", None)
167         indexing = getattr(global_config, "indexing", None)
168         return {
169             "status": "ok",
170             "sport": sport,
171             "default_segmenter": getattr(indexing, "default_segmenter", None),
172         }
173
174     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
175         """The full hash ? validate ? segment ? report pipeline for one video.
176
177         This is the former synchronous /api/process body, unchanged in behavior, now run on
178         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
179         the sync endpoint used to return (UI compatibility); the per-video observability
180         report is still emitted in `finally:` and grafted as response["reports"].
181
182         `progress` is an optional callable(stage: str) used to surface coarse job progress.
183         Raises on unexpected internal errors ? the caller records those as a job failure
184         (after this function has already restored the pre-run segments, see below).
185         """
186         notify = progress or (lambda stage: None)
187
188         notify("hashing")
189         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
190         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
191         # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
192         # reading # consumers (hash / validators / segmenter) use the resolved local path.
193         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
194         "storage", None))
195         video_id = compute_video_id(local_video)
196         was_reingest = db_instance.get_video(video_id) is not None
197
198         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
199         # report)
200         notify("validating")
201         logger.info(f"Running validations for {req.video_path}...")
202         val_results, val_context = registry.run_all_with_context(local_video, global_config)
203         failures = [r for r in val_results if not r.passed]
204
205         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
206         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
207         "motion")
208         seg_name = req.segmenter or default_seg
209         segmenter_actual = None
210         segmentation_ran = False
211         response: Optional[Dict[str, Any]] = None
212         try:
213             if failures:
214                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
215                 # status; it no longer destroys the video's prior good segments.
216                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
217                 for r in val_results])
218             response = {
219                 "video_id": video_id,

```

```

216         "status": "failed",
217         "message": "Video failed validation checks.",
218         "results": [r.model_dump() for r in val_results],
219     }
220     return response
221
222     # 2. Run segmenter & indexer
223     logger.info("[API] Video passed checks. Segmenting and indexing...")
224     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
225 r in val_results])
226
227     segmenter_cls = segmenter_registry.get(seg_name)
228     if not segmenter_cls:
229         # Submit-time validation already 400s unknown names; this is the defensive
230         # in-worker path (e.g. config default pointing at an unregistered
231 segmenter).
232         available = list(segmenter_registry.list_available().keys())
233         response = {
234             "video_id": video_id,
235             "status": "failed",
236             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
237 {available}",
238             "results": [r.model_dump() for r in val_results],
239             "play_segments": [],
240         }
241         return response
242
243     logger.info(f"[API] Using segmenter: {seg_name}")
244     notify(f"segmenting ({seg_name})")
245     segmenter_actual = seg_name
246     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
247 only
248     # if the run yields segments; otherwise drop the new partial rows and keep the
249 old.
250     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
251     segmenter = segmenter_cls(db_instance, global_config)
252     try:
253         result = segmenter.process_video(video_id, local_video, req.player_pool,
254 req.max_frames)
255     except Exception:
256         # A raising segmenter must not leave half-written rows: restore the pre-run
257         # state (same destroy-after-confirm contract as the Failure branch), then
258         let
259             # the job worker record the failure.
260             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
261             raise
262     segmentation_ran = True
263
264     if isinstance(result, Failure):
265         logger.error(f"[API] Segmenter failed: {result.message}")
266         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
267         response = {
268             "video_id": video_id,
269             "status": "failed",
270             "message": f"Segmenter '{seg_name}' failed: {result.message}",
271             "results": [r.model_dump() for r in val_results],
272             "play_segments": [],
273         }
274         return response
275
276     segments = result.value if hasattr(result, "value") else result
277     notify("finalizing")
278     _finalize_reingest(video_id, segments, play_wm, cand_wm)
279
280     response = {

```



```

274         "video_id": video_id,
275         "status": "success",
276         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
277         "results": [r.model_dump() for r in val_results],
278         "play_segments": segments,
279     }
280     return response
281 finally:
282     # Always emit the per-video observability report (next to the video, or to
283     # report.output_dir). Never raises. Surface the paths in the response.
284     paths = emit_indexer_report(
285         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
286         val_results=val_results, val_context=val_context,
287         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
288         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
289     )
290     if paths and isinstance(response, dict):
291         response["reports"] = paths
292
293
294 @app.post("/api/process", status_code=202)
295 def process_video(req: ProcessRequest, request: Request):
296     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
297
298     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
299     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
300     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
301     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
302     """
303     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
304     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
305     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
306     try:
307         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
308     except edge_auth.EdgeAuthError as e:
309         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
310
311     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
312     try:
313         validate_input_path(req.video_path, allowed_root=allowed_root)
314         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
315         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
316         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
317     except ValueError as e:
318         raise HTTPException(status_code=400, detail=str(e)) from e
319     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
320     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
321     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
322     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
323     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
324     # rejects a non-local URI as out-of-root.
325     if input_ref.backend == "local" and not os.path.exists(input_ref.key):

```

```

326         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}''")
327     if req.segmenter and not segmenter_registry.get(req.segmenter):
328         available = list(segmenter_registry.list_available().keys())
329         raise HTTPException(
330             status_code=400,
331             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
332         )
333
334     job_id = uuid.uuid4().hex
335     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
336                                 req.player_pool, req.max_frames, owner_id=owner_id):
337         raise HTTPException(status_code=500, detail="Failed to persist job record.")
338     _get_executor().submit(_drain_due_jobs)
339     return JSONResponse(
340         status_code=202,
341         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
342     )
343
344
345     @app.get("/api/jobs/{job_id}/cost")
346     def get_job_cost(job_id: str):
347         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
348         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
349         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
350         cost = db_instance.get_job_cost(job_id)
351         if cost is None:
352             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
353         fx = global_config.billing.fx_inr_per_usd
354         usd = cost.get("cost_usd")
355         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
356         cost["fx_inr_per_usd"] = fx
357         return cost
358
359
360     @app.get("/api/videos/{video_id}/cost")
361     def get_video_cost(video_id: str):
362         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
363         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
364         agg = db_instance.get_video_cost(video_id)
365         fx = global_config.billing.fx_inr_per_usd
366         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
367         agg["fx_inr_per_usd"] = fx
368         return agg
369
370
371     @app.get("/api/jobs/{job_id}")
372     def get_job(job_id: str):
373         """Job state: {status, progress, result, reports}. `status` = execution state
374         (queued|running|done|failed); the pipeline verdict is result["status"]."""
375         job = db_instance.get_job(job_id)
376         if not job:
377             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
378         result = job.get("result")
379         return {
380             "job_id": job["id"],
381             "status": job["status"],
382             "progress": job.get("progress"),
383             "video_id": job.get("video_id"),
384             "error": job.get("error"),

```

```

385         "result": result,
386         "reports": (result or {}).get("reports"),
387         "created_at": job.get("created_at"),
388         "started_at": job.get("started_at"),
389         "finished_at": job.get("finished_at"),
390     }
391
392     # --- Chunked upload (B2, PR-2) -----
393     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
394     # in
395     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
396     # via
397     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
398     # sequential-part logic, and abort cleanup ? is unchanged.
399     from backend.api import uploads as uploads_api
400
401     app.include_router(uploads_api.router)
402
403     # --- Highlight download (B3, PR-2) -----
404     @app.get("/api/highlights/{video_id}/{filename}")
405     def download_highlight(video_id: str, filename: str):
406         """Stream a compiled highlight reel ? `filename` is the bare name given to
407         /api/compile
408         (which only writes into output_dir; the browser previously got back an unreachable
409         server-local path)."""
410         try:
411             name = safe_output_filename(filename)
412             except ValueError as e:
413                 raise HTTPException(status_code=400, detail=str(e)) from e
414             if not db_instance.get_video(video_id):
415                 raise HTTPException(status_code=404, detail="Indexed video record not found.")
416             output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
417             or "output"
418             path = os.path.join(output_dir, name)
419             try:
420                 path = resolve_under_root(path, output_dir)
421             except ValueError as e:
422                 raise HTTPException(status_code=400, detail=str(e)) from e
423             if not os.path.isfile(path):
424                 raise HTTPException(status_code=404,
425                                     detail=f"No compiled file named '{name}'. Compile first via
426                                     /api/compile.")
427             media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
428             return FileResponse(path, media_type=media, filename=name)
429
430     @app.post("/api/query")
431     def query_play_segments(req: QueryRequest):
432         filters = {
433             "server": req.server,
434             "receiver": req.receiver,
435             "duration_min": req.duration_min,
436             "event_type": req.event_type
437         }
438         segments = db_instance.query_play_segments(req.video_id, filters)
439         return {"play_segments": segments}
440
441     @app.post("/api/compile")
442     def compile_highlights(req: CompileRequest):
443         video = db_instance.get_video(req.video_id)
444         if not video:
445             raise HTTPException(status_code=404, detail="Indexed video record not found.")

```

```

445     # Retrieve matching play segments from db
446     all_segments = db_instance.query_play_segments(req.video_id, {})
447     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
448
449     if not matched_segments:
450         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
451
452     # Reject path traversal / absolute output names (os.path.join would otherwise let an
453     # absolute or '..'-laden output_filename write anywhere on disk).
454     try:
455         out_name = safe_output_filename(req.output_filename)
456     except ValueError as e:
457         raise HTTPException(status_code=400, detail=str(e)) from e
458
459     # The configured shot-count floor (stitching.min_shot_count) applies on the API
460     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
461     # metadata are exempt: the floor filters known counts only, so explicitly
462     # requested manual/legacy segments can't silently vanish under the default floor.
463     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
464     kept = []
465     for s in matched_segments:
466         meta = s.get("metadata") or {}
467         sc = meta.get("shot_count") if isinstance(meta, dict) else None
468         try:
469             if sc is not None and int(sc) < stitching.min_shot_count:
470                 continue
471         except (TypeError, ValueError):
472             pass # unparseable count -> treat as unknown, keep
473         kept.append(s)
474     if len(kept) < len(matched_segments):
475         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
476                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
477         if not kept:
478             raise HTTPException(status_code=400,
479                                 detail=f"All {len(matched_segments)} matching segments fall
below "
480                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
481
482     # Extract start/end time pairs
483     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
484
485     # Run compiler with the configured stitching paddings + optional branding watermark
486     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
487     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
488                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
489     try:
490         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
491         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
492         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
493         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
494         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
495         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))

```

```

496         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
497         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
498         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
499         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
500         return {"status": "success", "filepath": out_path, "download_url": download_url}
501     except Exception as e:
502         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
503
504     # --- CLI Debug Utility & Orchestration ---
505     def run_cli_diagnose_clip(video_path: str, timestamp: float):
506         """CLI debugging utility to examine segment properties around a timestamp."""
507         print("=" * 60)
508         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
509         print("=" * 60)
510
511         cfg = load_config() # legacy wrapper still works, returns dict for now
512
513         # 1. Validation check diagnostics
514         print("[DIAGNOSTIC] Running validation framework...")
515         results = registry.run_all(video_path, cfg)
516         for r in results:
517             status_symbol = "[PASS]" if r.passed else "[FAIL]"
518             print(f" {status_symbol} {r.validator_name}: {r.message}")
519             if r.details:
520                 print(f"         Details: {r.details}")
521
522         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
523         print("-" * 60)
524         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
525         import cv2
526         cap = cv2.VideoCapture(video_path)
527         if not cap.isOpened():
528             print("[FAIL] OpenCV could not open video.")
529             return
530
531         fps = cap.get(cv2.CAP_PROP_FPS)
532         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
533
534         start_frame = max(0, int((timestamp - 3.0) * fps))
535         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
536
537         # Measure frame-by-frame changes in sample region
538         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
539         prev_gray = None
540         motions = []
541
542         for _ in range(start_frame, end_frame):
543             ret, frame = cap.read()
544             if not ret:
545                 break
546             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
547             gray = cv2.GaussianBlur(gray, (21, 21), 0)
548
549             if prev_gray is not None:
550                 diff = cv2.absdiff(prev_gray, gray)
551                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
552                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
553                 motions.append(motion_ratio)
554             prev_gray = gray
555
556         cap.release()

```

```

557
558     if motions:
559         avg_motion = sum(motions) / len(motions)
560         # Support both legacy dict (from wrapper) and future direct model
561         if hasattr(cfg, "indexing"):
562             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
563         else:
564             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
565         print(f" Sample Frame Count: {len(motions)}")
566         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
567         if avg_motion > threshold:
568             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
569         else:
570             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
571     else:
572         print(" [ERROR] No visual frames were extracted in the sample range.")
573
574     print("=" * 60)
575
576
577     def _enforce_startup_or_exit(config) -> None:
578         """M5: run the per-profile startup checks for the server/CLI path and
`sys.exit(1)` on a
579         FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
any work.
580         A strict no-op when no `deployment.profile` is selected (today's behaviour). No
GPU probe
581         here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
582         so the fail-fast wiring + the default-OFF no-op are unit-testable."""
583         try:
584             enforce_startup_checks(config, logger=logger)
585         except StartupCheckError as e:
586             logger.error("[STARTUP] refusing to start: %s", e)
587             sys.exit(1)
588
589
590     def main():
591         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
592         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
593         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
594         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
595         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
596         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
597         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
598         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
599
600         available_segmenters = list(segmenter_registry.list_available().keys())
601         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
602         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
603         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")

```

```

604     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
605
606     args = parser.parse_args()
607
608     if args.setup:
609         # Invoke the diagnostics/bootstrap script (renamed from the old root
610         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
611         # build system). Resolve its path from this file so `indexer --setup`
612         # works regardless of the caller's cwd.
613         import subprocess
614         doctor_path = os.path.join(
615             os.path.dirname(os.path.abspath(__file__)),
616             "scripts", "doctor.py",
617         )
618         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
619         subprocess.run([sys.executable, doctor_path])
620         return
621
622     if args.trim_start is not None and args.trim_end is not None:
623         if not args.video_path:
624             print("[ERROR] Please provide --video-path to extract a segment.")
625             return
626
627         # Determine output path
628         out_filename = args.trim_output or "trimmed_segment.mp4"
629         if os.path.isabs(out_filename):
630             out_path = out_filename
631             out_dir = os.path.dirname(out_path)
632             out_name = os.path.basename(out_path)
633         else:
634             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
635             out_path = os.path.join(out_dir, out_filename)
636             out_name = out_filename
637
638         os.makedirs(out_dir, exist_ok=True)
639         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
640
641         # global_config isn't initialized this early; load the typed config so the trim
642         # clip carries the same configured stitching paddings + watermark as a real
643         # highlight (watermark default-on).
644         stitching = load_app_config().stitching
645         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
646                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
647         try:
648             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
649             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
650         except Exception as e:
651             print(f"[ERROR] Failed to compile trimmed segment: {e}")
652         return
653
654     if args.debug_clip is not None:
655         if not args.video_path:
656             print("[ERROR] Please provide --video-path to debug a specific clip.")
657             return
658         run_cli_diagnose_clip(args.video_path, args.debug_clip)
659         return
660
661     # Initialize Global Config and DB
662     global global_config, db_instance
663     global_config = load_app_config() # returns typed AppConfig

```

```

664         _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
665
666         # The CLI --output-dir overrides the configured output directory. It now lives
667         # on the typed model (OutputConfig.output_dir), so every consumer ? including
668         # /api/compile ? reads the same value instead of a write-only runtime hack.
669         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671         global_config.output.output_dir = resolve_output_dir(args.output_dir)
672         os.makedirs(global_config.output.output_dir, exist_ok=True)
673
674         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
675         db_instance = Database(db_path)
676
677         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
678         # the executor is in-process. Mark them failed so pollers see a terminal state.
679         swept = db_instance.fail_stale_jobs()
680         if swept:
681             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
682
683         # CLI processing mode (no web UI needed)
684         if args.video_path and not args.server:
685             print(f"[CLI] Processing video file: {args.video_path}")
686             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
687             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
688             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
689             storage_cfg = getattr(global_config, "storage", None)
690             try:
691                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
692             except ValueError as e:
693                 print(f"[FAIL] {e}")
694                 return
695             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
696                 print(f"[FAIL] Video file not found: {args.video_path}")
697                 return
698             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
699
700             video_id = compute_video_id(local_video)
701             was_reingest = db_instance.get_video(video_id) is not None
702
703             # Validate (with-context variant surfaces ffprobe_meta for the report)
704             print("[CLI] Validating...")
705             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
706             failures = [r for r in val_results if not r.passed]
707
708             # Save video status
709             status = "failed" if failures else "processed"
710             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
711
712             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
713             segmenter_actual = None
714             segmentation_ran = False
715             try:
716                 print("\n" + "="*40)
717                 print("VALIDATION SUMMARY")
718                 print("="*40)

```



```

719         for r in val_results:
720             status_symbol = "[PASS]" if r.passed else "[FAIL]"
721             print(f"{status_symbol} {r.validator_name}: {r.message}")
722         print("="*40 + "\n")
723
724         if failures:
725             print("[FAIL] Video failed checks. Indexing halted.")
726             return
727
728         # Index
729         segmenter_cls = segmenter_registry.get(seg_name)
730         if not segmenter_cls:
731             print(f"[FAIL] Segmenter '{seg_name}' not found.")
732             return
733
734         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
735         segmenter_actual = seg_name
736         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
737         segmenter = segmenter_cls(db_instance, global_config)
738         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
739         segmentation_ran = True
740
741         if isinstance(result, Failure):
742             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
743             print("[FAIL] Indexing halted for video.")
744             segments = []
745         else:
746             segments = result.value if hasattr(result, "value") else result
747             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
748             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
749             _finalize_reingest(video_id, segments, play_wm, cand_wm)
750         finally:
751             # Always emit the per-video observability report. Never raises.
752             paths = emit_indexer_report(
753                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
754                 val_results=val_results, val_context=val_context,
755                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
756                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
757             )
758             if paths:
759                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
760                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
761                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
762             return
763
764         # Start Local Server Mode
765         if args.server or not args.video_path:
766             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
767             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
768             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
769             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
770             _get_executor().submit(_drain_due_jobs)
771             uvicorn.run(app, host="127.0.0.1", port=args.port)
772
773     if __name__ == "__main__":
774         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
775         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?

```

```

a SEPARATE
776     # module object. If `main()` runs here it configures `__main__`'s globals, leaving
777     # `backend.main.db_instance` = None; the async-job drain then crashes silently on
778     # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
779     # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
780     # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
781     import backend.main as _bm
782
783     _bm.main()
784

```

9. assistant (2026-06-21T14:42:32.753Z)

Now I have all five files. Let me adversarially analyze the behaviour-neutrality contract. The key question: with `RALLY\_APP\_HOME` unset, is everything byte-identical to pre-P0a?

Let me check the pre-P0a baseline by looking at git history to know exactly what the old behaviour was.

10. user (2026-06-21T14:42:35.062Z)

.gitattributes	10 +
.github/workflows/ci.yml	28 +
.gitignore	Bin 924 -> 1117 bytes
CLAUDE.md	19 +-
backend/api/__init__.py	1 +
backend/api/auth.py	127 +++
backend/api/job_worker.py	184 +++++
backend/api/uploads.py	155 ++++
backend/billing.py	60 ++
backend/compute/__init__.py	61 ++
backend/compute/base.py	53 ++
backend/compute/cloud_run.py	155 ++++
backend/config/__init__.py	26 +-
backend/config/loader.py	120 ++-
backend/config/models.py	173 ++++-
backend/config/presets.py	132 ++++
backend/config/startup.py	182 +++++
backend/eval/ablation.py	162 +--
backend/eval/ablation_pdf.py	141 ++++
backend/eval/experiment.py	82 ++
backend/eval/fusion_audio.py	43 +-
backend/eval/fusion_compare.py	39 +-
backend/eval/golden_fixtures.py	848 ++++++
backend/eval/golden_real_fixtures.py	343 ++++++
backend/eval/rally_seg_eval.py	415 ++++++-
backend/eval/tolerance_metrics.py	277 ++++++
backend/eval/windowing.py	41 +-
backend/infrastructure/database.py	717 ++++++-----
backend/main.py	468 +++++-----
backend/pipeline/detectors/court_gate.py	99 +++
backend/pipeline/detectors/device.py	56 ++
backend/pipeline/detectors/native_wasb_runner.py	263 ++++++
backend/pipeline/detectors/stub_runner.py	157 ++++
backend/pipeline/detectors/tracknet_runner.py	196 ++++-
backend/pipeline/detectors/wasb_infer.py	43 +-
backend/pipeline/segmenters/_keyhint.py	13 +
backend/pipeline/segmenters/fusion_hybrid.py	29 +-
backend/pipeline/segmenters/gemini.py	554 ++++++-----
backend/pipeline/segmenters/motion.py	30 +-
backend/pipeline/segmenters/trajectory_hybrid.py	68 +-

11. user (2026-06-21T14:42:35.289Z)

```
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
(#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
```

12. assistant (2026-06-21T14:42:38.734Z)

The diff is vs master, but this is a P0a worktree branched off something. Let me get the precise diff for the changed files to see exactly what P0a changed (so I can compare to pre-P0a behaviour) and verify the static-mount change which the task mentions.

13. user (2026-06-21T14:42:40.077Z)

```
diff --git a/backend/config/loader.py b/backend/config/loader.py
index dd34507..f937f23 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -16,6 +16,8 @@ from typing import Any

from pydantic import BaseModel

+from backend.utils.app_home import default_config_path
+
from .models import AppConfig
from .presets import (
    COMPUTE_TARGET_FOR_PROFILE,
@@ -27,7 +29,10 @@ from .presets import (

    logger = logging.getLogger(__name__)

-DEFAULT_CONFIG_PATH = Path("config.json")
+# Back-compat alias (was a module constant; no external consumers). Prefer
+# ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a). With the
+# env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
+DEFAULT_CONFIG_PATH = default_config_path()

def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
@@ -109,7 +114,7 @@ def load_config(path: str | Path | None = None) -> AppConfig:
    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
-    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+    cfg_path = Path(path) if path is not None else default_config_path()

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
@@ -199,7 +204,7 @@ def save_default_config(path: str | Path | None = None) -> Path:
    Used by scripts/doctor.py diagnostics and the --setup CLI path.
    Overwrites if the file exists.
    """
-    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+    cfg_path = Path(path) if path is not None else default_config_path()
    cfg = AppConfig()
    cfg_path.parent.mkdir(parents=True, exist_ok=True)
    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
diff --git a/backend/main.py b/backend/main.py
index 6768d30..8e931f8 100644
```

```

--- a/backend/main.py
+++ b/backend/main.py
@@ -7,7 +7,16 @@ from urllib.parse import quote
import uvicorn
from dotenv import load_dotenv

-load_dotenv()
+# PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins RALLY_APP_HOME),
+# else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
+# stdlib-only import ? keeps this pre-pydantic line light.
+from backend.utils.app_home import default_env_path, resolve_output_dir
+
+_env_path = default_env_path()
+if _env_path.exists():
+    load_dotenv(_env_path)
+else:
+    load_dotenv()

# Centralized logging setup for the indexer (hot paths use logger, CLI output uses print for
direct user-facing messages).
logging.basicConfig(
@@ -67,13 +76,17 @@ app.add_middleware(
    allow_headers=["*"],
)

-# Serves static frontend files directly (now under api/ per approved structure)
-os.makedirs("backend/api/static", exist_ok=True)
-app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
+# Serves static frontend files directly (now under api/ per approved structure).
+# PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from an
+# arbitrary directory finds the shipped assets instead of creating a stray
+./backend/api/static.
+# (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
+_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
+os.makedirs(_STATIC_DIR, exist_ok=True)
+app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

@app.get("/", response_class=HTMLResponse)
def serve_index():
-    index_path = "backend/api/static/index.html"
+    index_path = os.path.join(_STATIC_DIR, "index.html")
    if os.path.exists(index_path):
        with open(index_path, "r", encoding="utf-8") as f:
            return f.read()
@@ -618,7 +631,7 @@ def main():
    out_dir = os.path.dirname(out_path)
    out_name = os.path.basename(out_path)
    else:
-        out_dir = args.output_dir
+        out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical
when unset)
        out_path = os.path.join(out_dir, out_filename)
        out_name = out_filename

@@ -653,7 +666,9 @@ def main():
    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
-    global_config.output.output_dir = args.output_dir
+    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
+    # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
+    global_config.output.output_dir = resolve_output_dir(args.output_dir)
    os.makedirs(global_config.output.output_dir, exist_ok=True)

```

```

db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
diff --git a/scripts/doctor.py b/scripts/doctor.py
index b14bda8..309431b 100644
--- a/scripts/doctor.py
+++ b/scripts/doctor.py
@@ -49,12 +49,16 @@ def init_default_config():
    """
    try:
        from backend.config import save_default_config
+        from backend.utils.app_home import default_config_path
    except Exception as e:
        print(f"[ERROR] Could not import backend.config loader: {e}")
        print("[INFO] Falling back to no-op for config initialization.")
        return

-    config_path = os.path.join(REPO_ROOT, "config.json")
+    # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
+    # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root while
+    # the server read from the CWD, so --setup and the server disagreed once CWD ? repo root.
+    config_path = str(default_config_path())
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return

```

14. assistant (2026-06-21T14:42:46.533Z)

Now I have the precise before/after. Let me analyze each behaviour-neutrality concern carefully.

****Key analysis points:****

1. ****`resolve\_output\_dir("output")` when unset**** ? ``app_home()`` returns ``Path(".")``, so ``str(Path(".") / "output")``. Let me verify what this produces on both POSIX and Windows, because ``str(Path(".") / "output")`` is NOT necessarily ``"output"``.

2. ****`default\_config\_path()` when unset**** ? ``Path(".") / "config.json"``. Is ``str()`` of that ``"config.json"`` or ``"./config.json"`` / ``"./config.json"``?

3. The static-mount change: ``os.makedirs("backend/api/static")`` ? ``os.makedirs(_STATIC_DIR)`` where ``_STATIC_DIR`` is an absolute package-relative path. This is a behaviour CHANGE (mentioned in the task as intended), but let me verify it doesn't break anything when `CWD == repo root`.

Let me empirically test the critical ``Path(".")`` join behaviour on this Windows machine, and also check whether ``cfg_path.exists()`` behaves identically.

15. user (2026-06-21T14:42:50.546Z)

```

os.name = nt
Path("./config.json") -> WindowsPath('config.json') | str -> 'config.json'
== Path("config.json")? True
str(Path("./output")) -> 'output'
== "output"? True
empty -> str(Path("./output")) -> 'output'
Path("./.env") -> WindowsPath('.env') | str -> '.env'

```

16. assistant (2026-06-21T14:42:54.291Z)

On Windows, ``pathlib`` normalizes ``Path(".") / "x"`` to drop the leading ``.`` so ``str()`` is ``"output" / "config.json" / ".env"`` ? byte-identical. The test fixtures confirm the same. Let me verify the same holds on POSIX, since the contract claims both. Let me check using a POSIX path reasoning and the `PurePosixPath`.

17. user (2026-06-21T14:42:57.578Z)

```

POSIX Path("./config.json") -> PurePosixPath('config.json') | str -> 'config.json'

```

```
== PurePosixPath("config.json")? True
POSIX str(Path("./output") -> 'output' | == output? True
POSIX Path("./.env") -> PurePosixPath('.env') | str -> '.env'
```

18. assistant (2026-06-21T14:43:03.918Z)

Confirmed on both POSIX and Windows: `pathlib` collapses the ``.`` so the resolver outputs are byte-identical strings (`"config.json"`, `"output"`, `".env"`). The core resolver contract holds.

Now let me examine the subtler behaviour-change vectors:

1. `default_env_path().exists()` branch in `main.py` ? pre-P0a was always `load_dotenv()` (which searches CWD then walks up parent dirs). Now: if `Path(".env").exists()` (i.e., a `.env` exists in CWD), it calls `load_dotenv(Path(".env"))`. Otherwise falls back to `load_dotenv()`. This is a potential behaviour change. Let me think about the walk-up case.
2. `DEFAULT_CONFIG_PATH` evaluated at import ? if `RALLY_APP_HOME` is set AFTER import but config is loaded via `default_config_path()` (dynamic), the alias would diverge. But the alias has no consumers (confirmed by the diff comment). Let me verify there are truly no consumers.

19. user (2026-06-21T14:43:05.014Z)

```
C:\Users\avidu\Projects\bhi-wt-p0a\docs\CODE_MAP.md:132: - `::DEFAULT_CONFIG_PATH =
Path("config.json")` (? CWD-relative, NOT repo-root). `:_get_built_in_defaults` =
`AppConfig().model_dump(...)`
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:35:DEFAULT_CONFIG_PATH =
default_config_path()
```

20. user (2026-06-21T14:43:05.984Z)

```
C:\Users\avidu\Projects\bhi-wt-p0a\ruff.toml:17:# Entrypoint scripts deliberately run setup
(load_dotenv, logging config,
C:\Users\avidu\Projects\bhi-wt-p0a\tools\generate_highlights.py:34:from dotenv import
load_dotenv
C:\Users\avidu\Projects\bhi-wt-p0a\tools\generate_highlights.py:106:     load_dotenv()
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\run_process_and_stitch.py:4:from dotenv import
load_dotenv
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\run_process_and_stitch.py:6:load_dotenv()
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\run_phase3_eval.py:29:from dotenv import load_dotenv
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\run_phase3_eval.py:30:load_dotenv() # make
GEMINI_API_KEY (etc.) available to the AI-backed segmenters
C:\Users\avidu\Projects\bhi-wt-p0a\scripts\README.md:13:If a script needs `load_dotenv()` /
`sys.path` setup before importing `backend`, add
C:\Users\avidu\Projects\bhi-wt-p0a\docs\CODE_MAP.md:29:| Standalone run-driver / one-off entry
CLI (has __main__, NOT imported by app code) | scripts/ | e.g.
`scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as python
scripts/<name>.py. If it needs load_dotenv() / sys.path before importing backend, add
"scripts/<name>.py" = ["E402"] to ruff.toml. |
C:\Users\avidu\Projects\bhi-wt-p0a\docs\CODE_MAP.md:121:- @scripts/run_phase3_eval.py::main ?
walk manifest, segment-if-needed, score, aggregate. load_dotenv() at import. ? DB key = file
MD5, NOT manifest's video_id (kept as label); config via backend.config.load_config;
segmenter_cls(db, cfg) receives the typed AppConfig (same object the live pipeline feeds
segmenters).
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:8:from dotenv import load_dotenv
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:17:     load_dotenv(_env_path)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:19:     load_dotenv()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:20:the heavier pydantic-backed
`backend.config` package ? keeping the early load_dotenv path light.
```

21. assistant (2026-06-21T14:43:15.241Z)

Confirmed:

- `DEFAULT_CONFIG_PATH` has no runtime consumers (only the loader's own def + a doc reference).

The alias is dead weight but harmless. However, ``docs/CODE_MAP.md:132`` still documents it as ``Path("config.json")`` literal ? a stale doc claim now that it's ``default_config_path()``. Minor doc-staleness, not a behaviour issue.

Now the important one ? **the ``load_dotenv`` change**. Let me analyze the pre-P0a vs post-P0a behaviour precisely.

Pre-P0a: ``load_dotenv()`` with no args. python-dotenv's default behaviour: it calls ``find_dotenv()`` which **searches the CWD and walks UP parent directories** for a ``.env`` file, using the caller's frame to start. It returns the first ``.env`` found walking up.

Post-P0a (env unset): ``_env_path = Path(".env")``. If ``Path(".env").exists()`` (relative to CWD) ? ``load_dotenv(Path(".env"))``. Else ? ``load_dotenv()`` (walk-up).

So the divergence: **if a ``.env`` exists in CWD, pre-P0a ``load_dotenv()`` would also find it first (walk-up starts at CWD), so identical. If NO ``.env`` in CWD, both fall to ``load_dotenv()`` walk-up ? identical.**

But there's a subtle gap. ``load_dotenv()`` (no path) uses ``find_dotenv(usecwd=False)`` by default, which starts searching from the **directory of the calling frame's file**, NOT necessarily the CWD. Let me verify python-dotenv's actual default-search semantics, because this is the crux of whether the CWD ``.env`` branch changes behaviour.

22. user (2026-06-21T14:43:16.852Z)

dotenv version: ?

```
def find_dotenv(
    filename: str = ".env",
    raise_error_if_not_found: bool = False,
    usecwd: bool = False,
) -> str:
    """
    Search in increasingly higher folders for the given file

    Returns path to the file if found, or an empty string otherwise
    """

    def _is_interactive():
        """Decide whether this is running in a REPL or IPython notebook"""
        if hasattr(sys, "ps1") or hasattr(sys, "ps2"):
            return True
        try:
            main = __import__("__main__", None, None, fromlist=["__file__"])
        except ModuleNotFoundError:
            return False
        return not hasattr(main, "__file__")

    def _is_debugger():
        return sys.gettrace() is not None

    if usecwd or _is_interactive() or _is_debugger() or getattr(sys, "frozen", False):
        # Should work without __file__, e.g. in REPL or IPython notebook.
        path = os.getcwd()
    else:
        # will work for .py files
        frame = sys._getframe()
        current_file = __file__

        while frame.f_code.co_filename == current_file or not os.path.exists(
            frame.f_code.co_filename
        ):
            assert frame.f_back is not None
            frame = frame.f_back
        frame_filename = frame.f_code.co_filename
        path = os.path.dirname(os.path.abspath(frame_filename))
```

```

for dirname in _walk_to_root(path):
    check_path = os.path.join(dirname, filename)
    if _is_file_or_fifo(check_path):
        return check_path

if raise_error_if_not_found:
    raise IOError("File not found")

return ""

```

23. assistant (2026-06-21T14:43:24.993Z)

This is the crux. ``load_dotenv()`` (pre-P0a) calls ``find_dotenv()`` with ``usecwd=False``. In a `**normal`.py` run**` (not REPL/notebook/debugger/frozen), ``find_dotenv`` does NOT start from CWD ? it starts from `**the directory of the calling frame's file**` (``backend/main.py``'s directory, i.e. ``backend/``) and walks UP from there. It does NOT search the CWD branch unless CWD happens to be an ancestor of ``backend/``.

This means there IS a real behaviour difference in the unset path. Let me construct the precise scenario:

```

- Pre-P0a: `load_dotenv()` ? `find_dotenv(usecwd=False)` ? starts at
`dirname(backend/main.py)` = `<repo>/backend`, walks up: `<repo>/backend`, `<repo>`, ... So a
`env` in the repo root is found even if CWD is elsewhere. A `env` in the CWD (when CWD
is NOT an ancestor of `backend/`) is NOT found.

- Post-P0a (unset): `_env_path = Path(".env")` resolved against CWD. If a `env` exists in
CWD ? `load_dotenv(Path(".env"))` loads the CWD `env`, NOT walking up to the frame dir.

```

So the behaviours diverge in two distinct scenarios. Let me empirically prove this rather than reason about it.

24. user (2026-06-21T14:43:26.830Z)

```

load_dotenv signature: (dotenv_path: Union[str, ForwardRef('os.PathLike[str]'), NoneType] =
None, stream: Optional[IO[str]] = None, verbose: bool = False, override: bool = False,
interpolate: bool = True, encoding: Optional[str] = 'utf-8') -> bool
def load_dotenv(
    dotenv_path: Optional[StrPath] = None,
    stream: Optional[IO[str]] = None,
    verbose: bool = False,
    override: bool = False,
    interpolate: bool = True,
    encoding: Optional[str] = "utf-8",
) -> bool:
    """Parse a .env file and then load all the variables found as environment variables.

```

Parameters:

```

    dotenv_path: Absolute or relative path to .env file.
    stream: Text stream (such as `io.StringIO`) with .env content, used if
        `dotenv_path` is `None`.
    verbose: Whether to output a warning the .env file is missing.
    overr

```

25. assistant (2026-06-21T14:43:30.268Z)

Confirmed: ``load_dotenv()`` with ``dotenv_path=None`` calls ``find_dotenv()`` (``usecwd=False``), which starts from the `**caller's file directory**` (``backend/``), not CWD. Let me now empirically demonstrate the two divergence scenarios with a controlled reproduction. I'll set up temp dirs to simulate.

26. user (2026-06-21T14:43:34.474Z)

setup done

27. user (2026-06-21T14:43:39.956Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 5, in <module>
    from pkg import mainmod
ModuleNotFoundError: No module named 'pkg'
```

28. assistant (2026-06-21T14:43:43.651Z)

The ``-c`` invocation makes ``find_dotenv`` treat it as interactive (no ``__main__.__file__``), which would skew the test anyway. Let me write a proper script file so ``find_dotenv``'s frame-walk behaves like a real ``py`` run, and import via a path-based loader.

29. user (2026-06-21T14:43:46.383Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\dotenv_test\runner.py", line 3, in <module>
    from pkg import mainmod
ModuleNotFoundError: No module named 'pkg'
```

30. assistant (2026-06-21T14:43:49.398Z)

Path translation issue between Git Bash and Windows Python. Let me use Windows-style paths in the runner.

31. user (2026-06-21T14:43:52.611Z)

```
PRE-P0a (bare load_dotenv): FROM_CWD = None
POST-P0a (CWD .env branch) : FROM_CWD = 'cwd_value'
CWD = C:\Users\avidu\AppData\Local\Temp\dotenv_test\cwd
module file dir = C:\Users\avidu\AppData\Local\Temp\dotenv_test\repo\pkg
```

32. assistant (2026-06-21T14:44:00.392Z)

****Empirically confirmed ? there IS a real behaviour change in the unset path.**** When a ``env`` file exists in the CWD but the CWD is *not* an ancestor of ``backend/main.py``'s directory:

```
- **PRE-P0a:** `load_dotenv()` ? `find_dotenv(usecwd=False)` walks up from `backend/`, never finds the CWD `env` ? `FROM_CWD = None`.
- **POST-P0a (env unset):** `Path(".env").exists()` is True (CWD `env`) ? `load_dotenv(Path(".env"))` loads it ? `FROM_CWD = 'cwd_value'`.
```

This ****violates the HARD CONTRACT**** ("env unset => byte-identical to pre-P0a"). The new code now loads a CWD ``env`` that pre-P0a would have ignored.

Now let me check the reverse divergence: a ``env`` in the repo root (frame-dir ancestor) but ****none in CWD****, with CWD elsewhere. Pre-P0a finds the repo-root ``env`` (walk-up). Post-P0a: ``Path(".env").exists()`` is False (CWD has none) ? falls to ``load_dotenv()`` ? same walk-up. So that case is ****identical****. The divergence is one-directional: only when a CWD ``env`` exists that the frame-walk wouldn't have found.

Let me confirm the reverse case (no CWD ``env``, repo-root ``env``) is identical, to be precise and fair.

33. user (2026-06-21T14:44:03.855Z)

```
PRE-P0a FROM_REPO = 'repo_value'
POST-P0a FROM_REPO = 'repo_value'
```

34. assistant (2026-06-21T14:44:15.338Z)

As predicted: the reverse case (repo-root ``env``, no CWD ``env``) is identical. The divergence is

strictly the ****new CWD-`.env` precedence****.

Now, how material is this in practice? In the ****standard dev/CI/repo-root run****, CWD == repo root, which IS the frame-dir ancestor (`backend/` is under repo root). So `find\_dotenv` from `backend/` walks up to repo root and finds the repo-root `.env` ? the same file `Path(".env")` resolves to. So ****in the canonical repo-root invocation the two are identical.**** The divergence only manifests when CWD ? a parent of `backend/` AND a `.env` sits in that CWD.

For a normal source-tree run that's an edge case, but for the ****packaged-app scenario this PR explicitly targets**** (launched from an arbitrary CWD), it's exactly the live scenario ? except in packaging the launcher sets `RALLY\_APP\_HOME`, so the unset branch wouldn't run. Still, the contract says "unset => byte-identical," and it is measurably not, so I'll flag it as a major (real, demonstrated contract violation; low real-world blast radius).

There's also a second-order subtlety: `load\_dotenv()` default does NOT override already-set env vars (`override=False`), and so does `load\_dotenv(path)` ? both default `override=False`, so no precedence change there. The only change is **which file** is discovered.

Now let me verify two more things:

1. The ****static-mount change**** ? does the new `\_STATIC\_DIR` resolve to the same dir as the old `"backend/api/static"` when run from repo root? And is the `index.html` read path consistent? This is an intended change but let me confirm it's behaviour-neutral when CWD==repo root and there are no other readers of the old literal path.
2. Whether anything else reads `"backend/api/static"` as a CWD-relative literal that now mismatches.

35. user (2026-06-21T14:44:17.354Z)

```
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:81:# arbitrary directory finds the shipped
assets instead of creating a stray ./backend/api/static.
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:83:_STATIC_DIR =
os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:93: return "<h3>Sports Highlight Indexer
Server is Running. Please create index.html in backend/api/static/</h3>"
```

36. user (2026-06-21T14:44:20.176Z)

```
old (CWD=repo root): C:\Users\avidu\Projects\bhi-wt-p0a\backend\api\static
new (package-rel) : C:\Users\avidu\Projects\bhi-wt-p0a\backend\api\static
same when CWD==repo root? True
```

37. assistant (2026-06-21T14:44:28.897Z)

The static-mount change is behaviour-neutral when CWD == repo root (the canonical dev/CI case), and there are no other readers of the old literal `"backend/api/static"` path. The static change is sound and is an **intended** fix (independent of `RALLY\_APP\_HOME`).

Now let me check the test file for an important gap relevant to the contract verification: does the test suite actually cover the `load\_dotenv` CWD-`.env` precedence change? And does `resolve\_output\_dir` have a subtle issue with `output\_dir` being whitespace or a path like `".output"`? Let me also check: is `app\_home()` cached or re-read each call (matters for the test monkeypatch and for the import-time `DEFAULT\_CONFIG\_PATH`)?

`app\_home()` reads `os.environ` on every call (not cached) ? good, dynamic. But `DEFAULT\_CONFIG\_PATH` is import-time frozen. Let me confirm whether any test or code relies on the frozen value vs dynamic, and check the loader test integration covers the unset==CWD path properly.

Let me also verify there are no other CWD-relative readers that P0a missed (the contract is about completeness of the abstraction too, but the lens here is strictly behaviour-neutrality when unset, so those are not regressions). Let me run the new test file and the loader tests to confirm green.

38. user (2026-06-21T14:44:30.964Z)

.....
19 passed in 0.29s

[100%]

39. assistant (2026-06-21T14:44:36.141Z)

All 19 pass. Now let me verify the dotenv divergence does not have a test guarding the contract (it doesn't ? the test file has no `load\_dotenv` test), confirming the regression is untested. Let me also double check one more potential issue: `resolve\_output\_dir` with a leading-dot relative path like `"/output"` ? `os.path.isabs("/output")` is False, so it goes `str(app\_home() / "/output")`. When unset, `str(Path(".") / "/output")`. Let me verify that's still byte-identical to the old `args.output\_dir` passthrough (old code just used `args.output\_dir` verbatim).

40. user (2026-06-21T14:44:39.294Z)

```
<string>:6: SyntaxWarning: invalid escape sequence '\s'
'output'      old='output'      new='output'      identical=True
'./output'    old='./output'    new='output'      identical=False
'a/b'         old='a/b'         new='a\\b'        identical=False
'out\\sub'    old='out\\sub'    new='out\\sub'    identical=True
'.'           old='.'           new='.'           identical=True
'output/'     old='output/'     new='output'      identical=False
```

41. assistant (2026-06-21T14:44:52.804Z)

Found a second behaviour change in the unset path. `resolve\_output\_dir` is NOT a pure identity for relative inputs ? `pathlib` normalizes them:

- `"/output"` ? `"output"` (leading `./` stripped)
- `"a/b"` ? `"a\b"` on Windows (forward slash ? backslash separator)
- `"output/"` ? `"output"` (trailing slash stripped)

Old code (`out\_dir = args.output\_dir`) used the string **verbatim**. New code routes it through `str(Path(".") / v)` which normalizes separators and strips redundant components. So a user passing `--output-dir ./myout` or `--output-dir a/b` (forward slashes on Windows) gets a **different string** stored in `global\_config.output.output\_dir` and used for `os.makedirs` + `db\_path` join.

Now, does this actually matter functionally? `os.makedirs("a/b")` and `os.makedirs("a\b")` create the same directory on Windows (both separators accepted), and `os.path.join` works either way. So the **filesystem effect** is the same. BUT:

1. The **stored config value** `global\_config.output.output\_dir` differs as a string, and that string is returned by `/api/config`, embedded in reports, and used to build `download\_url`/path-comparison logic. `"a/b"` vs `"a\b"` is an observable difference in API output.
2. The test `test\_resolve\_output\_dir\_unset\_is\_identity\_for\_relative` only checks `"output"` and `"custom"` ? single-component names that happen to be normalization-invariant. It does **not** test `./output`, `a/b`, or trailing slash, so it gives false confidence that the unset path is "identity."

The CWD-relative **single-segment** default (`"output"`) is fine ? which is the overwhelmingly common case (the argparse default is `"output"`). But the contract says "byte-identical," and for multi-segment / dotted / forward-slash / trailing-slash inputs it is not byte-identical to the verbatim string. Let me classify this as **minor** (functionally equivalent on disk; only the stored/echoed string differs, and only for non-default `--output-dir` values).

Let me do a final verification on `default\_os\_app\_home` correctness (Windows APPDATA path) and check whether `app\_home()` `expanduser()` could change the unset path ? it doesn't (unset returns `Path(".")` directly, no expanduser). And confirm `app\_path` isn't used anywhere in a way that changes unset behaviour.

42. user (2026-06-21T14:44:54.174Z)

C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:13:from backend.utils.app_home import

```

default_env_path, resolve_output_dir
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:15:_env_path = default_env_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:634:         out_dir =
resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical when unset)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:670:         # unset, resolve_output_dir("output")
== "output" (CWD-relative) ? byte-identical to before.
C:\Users\avidu\Projects\bhi-wt-p0a\backend\main.py:671:         global_config.output.output_dir =
resolve_output_dir(args.output_dir)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:19:from backend.utils.app_home
import default_config_path
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:33:# ``default_config_path()`` ? it
honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a). With the
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:35:DEFAULT_CONFIG_PATH =
default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:117:         cfg_path = Path(path) if
path is not None else default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:207:         cfg_path = Path(path) if
path is not None else default_config_path()
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:36:def app_home() -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:48:def app_path(*parts: str |
os.PathLike[str]) -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:49:         """A path under
:func:`app_home` (e.g. ``app_path("output")``)."""
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:50:         return
app_home().joinpath(*[str(p) for p in parts])
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:53:def default_config_path() ->
Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:58:         return app_home() /
"config.json"
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:61:def default_env_path() -> Path:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:63:         return app_home() / ".env"
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:66:def
resolve_output_dir(output_dir: str) -> str:
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:69:         With ``RALLY_APP_HOME``
unset, ``resolve_output_dir("output") == "output"`` (CWD-relative),
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:74:         return str(app_home() /
"output")
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:77:         return str(app_home() /
output_dir)
C:\Users\avidu\Projects\bhi-wt-p0a\backend\utils\app_home.py:80:def
default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:

```

43. assistant (2026-06-21T14:44:59.182Z)

The consumers are all accounted for. `app\_path` has no production consumers yet (only the test), so no unset-behaviour impact there.

Let me do a final sanity check on the `\_env\_path.exists()` call: it runs at `**import time**` of `backend.main`. In tests that import `backend.main`, this could read a CWD `.env` if one exists in the test CWD. But that's a test-environment concern, not the unset-contract per se. The full suite passed (1221), so no test CWD `.env` exists. This is consistent.

Let me clean up the temp test dir and finalize.

44. user (2026-06-21T14:44:59.974Z)

cleaned

45. assistant (2026-06-21T14:45:04.405Z)

I have completed my adversarial verification. Let me summarize the findings.

The core resolver contract holds: `default_config_path()`, `default_env_path()`, and `resolve_output_dir("output")` are all byte-identical to pre-P0a when `RALLY_APP_HOME` is unset,

on **both** POSIX and Windows (pathlib collapses the ``Path(".")`` prefix). But I found two demonstrated behaviour changes in the unset path that contradict the "byte-identical" hard contract.

46. user (2026-06-21T14:45:38.658Z)

Structured output provided successfully

47. assistant (2026-06-21T14:45:47.139Z)

I completed the adversarial behaviour-neutrality review of P0a and returned findings via StructuredOutput.

Summary of what I verified and found:

Core contract holds (verified on both Windows and POSIX): ``default_config_path()``, ``default_env_path()``, and ``resolve_output_dir("output")`` all produce byte-identical strings to pre-P0a when ``RALLY_APP_HOME`` is unset, because pathlib collapses the ``Path(".")`` prefix. The package-relative ``_STATIC_DIR`` static mount is byte-identical when `CWD==repo root` and has no other readers of the old literal.

Two demonstrated contract violations in the unset path:

- **MAJOR** (``backend/main.py:15-19``): the new ``load_dotenv`` branch loads a CWD ``.env`` that pre-P0a's bare ``load_dotenv()`` (which uses ``find_dotenv(usecwd=False)``, walking up from ``backend/``, not CWD) would have ignored. I reproduced this directly ? pre-P0a left the var ``None``, P0a loaded it. The canonical repo-root run is unaffected, but the contract is about the general unset case.
- **MINOR** (``backend/utils/app_home.py:77``): ``resolve_output_dir`` normalizes relative inputs (``.output`?```output``, ``a/b`?```a\b`` on Windows, ``output/`?```output``) whereas the old code stored ``args.output_dir`` verbatim ? functionally equivalent on disk, but the stored/echoed string (``/api/config``, reports) differs for non-default ``--output-dir`` values.

Two nits: the "identity for relative" test only exercises normalization-invariant single-segment names (false confidence), and the import-frozen ``DEFAULT_CONFIG_PATH`` alias plus a stale ``docs/CODE_MAP.md:132`` line.

The recommended fix for the major: prefer an app-home ``.env`` only when ``RALLY_APP_HOME`` is actually set, falling through to the verbatim pre-P0a ``load_dotenv()`` when unset.